Przegląd

Workflow Templates to gotowe do importu pliki JSON definiujące kompletne workflowy automatyzacji marketingowej. System umożliwia użytkownikom jednym kliknięciem załadowanie predefiniowanego workflowu, który zostaje zduplikowany na ich koncie i może być dowolnie modyfikowany.

Architektura przepływu

```
sequenceDiagram
    participant UI as Angular Frontend
    participant Assets as Static Assets
    participant API as Laravel API
    participant DB as Database


    UI->>Assets: GET /assets/workflow-templates/{slug}.json
    Assets-->>UI: JSON template file
    UI->>API: POST /api/v1/workflow (name)
    API-->>UI: { workflow_id }
    UI->>API: POST /api/v1/workflow/{id}/import (JSON file)
    API->>DB: Utwórz węzły, konfigurację, globals
    API-->>UI: { success }
    UI->>UI: Przekieruj do /workflows/{id}/edit
```

Lokalizacja plików

Lokalizacja	Opis
`sembot-angular/src/assets/workflow-templates/`	Pliki JSON ładowane przez frontend
`sembot-laravel/workflow-templates/`	Kopie referencyjne + dokumentacja backendowa

Format JSON szablonu

Minimalna struktura

```
{
  "data": {
    "name": "Nazwa Szablonu",
    "description": "Opis szablonu",
    "status": "inactive",
    "globals": {},
    "projects": [],
    "nodes": []
  }
}
```

Kompletna struktura

```
{
  "data": {
    "name": "string — wyświetlana nazwa workflowu",
    "description": "string — opis dla użytkownika (może zawierać instrukcje konfiguracji)",
    "status": "inactive | active",
    "globals": {
      "nazwa_zmiennej": {
        "value": 50,
        "type": "number | string | boolean"
      }
    },
    "projects": [],
    "nodes": [
      {
        "id": 1,
        "index": 1,
        "parent_id": null,
        "type": "trigger | getter | flow_control | action | ai_agent",
        "payload": {}
      }
    ]
  }
}
```

Pole `globals`

Zmienne globalne dostępne w wyrażeniach jako `{globals.nazwa_zmiennej}`. Użytkownik uzupełnia je przed aktywacją workflowu.

```
"globals": {
  "cpa_target": { "value": 50, "type": "number" },
  "min_spend": { "value": 100, "type": "number" },
  "alert_email": { "value": "", "type": "string" }
}
```

Struktura węzła

Pole	Typ	Opis
`id`	int	Unikalny identyfikator węzła w obrębie workflowu
`index`	int	Kolejność wykonania (używana w wyrażeniach jako prefiks `{N.pole}`)
`parent_id`	int \	null	ID węzła nadrzędnego (null = węzeł główny)
`type`	string	Typ węzła: `trigger`, `getter`, `flow_control`, `action`, `ai_agent`
`payload`	object	Konfiguracja specyficzna dla typu

Typy węzłów

trigger

Definiuje wyzwalacz workflowu.

Harmonogram (schedule)

```
{
  "type": "trigger",
  "payload": {
    "type": "schedule",
    "data": {
      "timezone": "Europe/Warsaw",
      "start_date": "2026-01-05",
      "start_time": "08:00",
      "end_date_type": "never",
      "end_date": null,
      "occurrences": null,
      "repeat": {
        "frequency": "weekly | daily | hourly | monthly",
        "interval": 1,
        "days_of_week": [1],
        // Integer: 0=Sunday, 1=Monday, ... 6=Saturday — NOT string values
        "days_of_month": "first_day",
        "month_date": null,
        "time": "08:00"
      }
    }
  }
}
```

Wartości `frequency`:

Wartość	Opis	Przykład użycia
`weekly`	Co N tygodni, wybrane dni	Poniedziałkowe raporty
`daily`	Co N dni	Codzienne sprawdzenie budżetu
`hourly`	Co N godzin	Budget Protector co 4h
`monthly`	Co N miesięcy	Miesięczny przegląd

---

getter

Pobiera dane z zewnętrznych integracji. Wszystkie gettery wymagają `connection_id` i `project_id` (uzupełniane przez użytkownika, w szablonach: `null`).

Google Ads — kampanie

```
{
  "type": "getter",
  "payload": {
    "type": "get_google_campaign",
    "data": {
      "campaign_source": "google",
      "resource_type": "campaigns",
      "connection_id": null,
      "project_id": null,
      "campaign_statuses": ["ENABLED"]
    }
  }
}
```

Pola wyjściowe: `campaignId`, `campaignName`, `campaignStatus`, `campaignType`, `biddingStrategyType`

Google Ads — metryki kampanii

```
{
  "type": "getter",
  "payload": {
    "type": "get_google_ads_metrics",
    "data": {
      "connection_id": null,
      "project_id": null,
      "resource": "campaign",
      "start_date": "{NOW().subtractNumberOfDays(30).formatDateTime('Y-m-d')}",
      "end_date": "{NOW().formatDateTime('Y-m-d')}",
      "resourceId": "{N.campaignId}",
      "limit": 1
    }
  }
}
```

Pola wyjściowe: `cost`, `clicks`, `impressions`, `ctr`, `conversions`, `conversions_value`, `costPerConversion` (alias: `cpa`)

Google Ads — wyniki szczegółowe (performance)

```
{
  "type": "getter",
  "payload": {
    "type": "get_google_ads_performance",
    "data": {
      "connection_id": null,
      "project_id": null,
      "resource": "keyword_view | search_term_view | campaign",
      "start_date": "{NOW().subtractNumberOfDays(60).formatDateTime('Y-m-d')}",
      "end_date": "{NOW().formatDateTime('Y-m-d')}",
      "limit": 100
    }
  }
}
```

Wartości `resource`: `campaign`, `keyword_view`, `search_term_view`, `ad_group`

Pola wyjściowe per resource:

resource	Pola
`campaign`	`campaignName`, `cost`, `clicks`, `impressions`, `ctr`, `conversions`, `roas`
`keyword_view`	`title` (tekst słowa), `campaign_name`, `ad_group_name`, `cost`, `clicks`, `conversions`, `ctr`
`search_term_view`	`title` (fraza), `campaign_name`, `impressions`, `clicks`, `conversions`, `ctr`

> ***Uwaga:*** *Parametry daty: `start_date` / `end_date` (snake_case), identycznie jak MS Ads. Pole `resource` wymagane; brak go powoduje błąd silnika.*

Google Ads — treści reklam

```
{
  "type": "getter",
  "payload": {
    "type": "get_google_ads_ad_copy",
    "data": {
      "connection_id": null,
      "project_id": null,
      "limit": 500
    }
  }
}
```

Pola wyjściowe (`GoogleAdsAdCopyDTO`):

Pole	Typ	Opis
`campaignId`	string	ID kampanii
`campaignName`	string	Nazwa kampanii
`adGroupId`	string	ID grupy reklam
`adGroupName`	string	Nazwa grupy reklam
`adId`	string	ID reklamy
`adType`	string	Typ reklamy (np. responsive search ad)
`adStatus`	string	Status: `ENABLED`, `PAUSED`, `REMOVED`
`headline1`	string	Nagłówek 1
`headline2`	string	Nagłówek 2
`headline3`	string	Nagłówek 3
`description1`	string	Opis 1
`description2`	string	Opis 2
`finalUrl`	string	Końcowy URL reklamy

> *Getter domyślnie pomija reklamy ze statusem `REMOVED`. Filtruj po `adStatus = ENABLED` żeby przetwarzać tylko aktywne.*

Google Merchant Center — top produkty

```
{
  "type": "getter",
  "payload": {
    "type": "get_google_merchant_center_top_products",
    "data": {
      "connection_id": null,
      "project_id": null,
      "report_granularity": "WEEKLY",
      "report_date": "{NOW().formatDateTime('Y-m-d')}"
    }
  }
}
```

Pola wyjściowe: `productClusterTitle`, `productClusterBrand`, `bestSellersRank`, `bestSellersRelativeDemand`, `bestSellersRelativeDemandChange`, `productClusterInventoryStatus`

> ***Ważne:*** *Getter zwraca dane rankingowe bestsellerów rynkowych (Google Trends dla Shopping), nie dane własnego inwentarza. Nie zawiera pól `availability`, `conversions`, `price`, `offerId` — te pola nie istnieją w tym getterze.*

Microsoft Ads — kampanie (z metrykami)

```
{
  "type": "getter",
  "payload": {
    "type": "get_microsoft_advertising_campaigns",
    "data": {
      "connection_id": null,
      "project_id": null,
      "start_date": "{NOW().subtractNumberOfDays(30).formatDateTime('Y-m-d')}",
      "end_date": "{NOW().formatDateTime('Y-m-d')}"
    }
  }
}
```

Pola wyjściowe (`CampaignDTO`):

Pole	Typ	Opis
`campaignId`	int	ID kampanii
`campaignName`	string	Nazwa kampanii
`campaignStatus`	string	Status kampanii
`spend`	float	Wydatki w wybranym okresie
`impressions`	int	Liczba wyświetleń
`clicks`	int	Liczba kliknięć
`ctr`	float	Click-through rate
`averageCpc`	float	Średni CPC
`conversions`	int	Liczba konwersji
`costPerConversion`	float	Koszt za konwersję (CPA)
`revenue`	float	Przychód z konwersji
`roas`	float	Return on Ad Spend
`impressionSharePercent`	float	% wyświetleń w stosunku do możliwych

> ***Uwaga:*** `get_microsoft_advertising_campaigns` *zwraca metryki* ***w jednym węźle*** *— nie wymaga osobnego gettera metryk jak Google Ads.*

Microsoft Ads — frazy wyszukiwania

```
{
  "type": "getter",
  "payload": {
    "type": "get_microsoft_advertising_search_terms",
    "data": {
      "connection_id": null,
      "project_id": null,
      "start_date": "{NOW().subtractNumberOfDays(30).formatDateTime('Y-m-d')}",
      "end_date": "{NOW().formatDateTime('Y-m-d')}"
    }
  }
}
```

Pola wyjściowe (`SearchTermDTO`): `searchQuery`, `keyword`, `keywordId`, `campaignName`, `campaignId`, `adGroupName`, `adGroupId`, `impressions`, `clicks`, `ctr`, `spend`, `conversions`, `costPerConversion`, `revenue`, `roas`, `averageCpc`

Microsoft Ads — słowa kluczowe

```
{
  "type": "getter",
  "payload": {
    "type": "get_microsoft_advertising_keywords",
    "data": {
      "connection_id": null,
      "project_id": null,
      "start_date": "{NOW().subtractNumberOfDays(60).formatDateTime('Y-m-d')}",
      "end_date": "{NOW().formatDateTime('Y-m-d')}"
    }
  }
}
```

Pola wyjściowe (`KeywordDTO`): `keyword`, `keywordId`, `campaignName`, `campaignId`, `adGroupName`, `adGroupId`, `spend`, `impressions`, `clicks`, `ctr`, `conversions`, `costPerConversion`, `revenue`, `roas`, `averageCpc`, `currentMaxCpc`

Phrase Monitor — monitoring widoczności

```
{
  "type": "getter",
  "payload": {
    "type": "get_phrase_monitor_competitors",
    "data": {
      "project_id": null,
      "monitor_type": "{globals.monitor_type}",
      "sort_order": "ASC"
    }
  }
}
```

Pola wyjściowe: `query`, `rank_absolute`, `domain`, `url`, `is_my_result`

> ***Uwaga:*** *W warunkach filtra używaj camelCase:* `rankAbsolute`, `isMyResult`, `query`, `domain`, `url`.

---

flow_control

Węzły sterujące przepływem danych.

filter — filtrowanie tablicy

```
{
  "type": "flow_control",
  "payload": {
    "type": "filter",
    "data": {
      "label": "Opcjonalna etykieta",
      "conditions": [
        {
          "left_operand_type": "data_field | user_input",
          "left_operand_value": "spend",
          "left_source": "2",
          "operator": "> | < | = | != | >= | <= | contains | not_contains | starts_with | ends_with",
          "right_operand_type": "user_input",
          "right_operand_value": "{globals.min_spend}",
          "right_source": null,
          "logical_operator": "AND | OR"
        }
      ],
      "create_additional_array": false
    }
  }
}
```

Typy operandów:

- `data_field` — odwołanie do pola z poprzedniego węzła (podaj `left_source` = ID węzła)
- `user_input` — wartość dosłowna lub wyrażenie dynamiczne

`create_additional_array`:

- `false` — przechodzi do kolejnego węzła dla każdego elementu spełniającego warunek
- `true` — tworzy nową tablicę przefiltrowanych elementów

iterator — pętla po elementach

```
{
  "type": "flow_control",
  "payload": {
    "type": "iterator",
    "data": {
      "label": "Opcjonalna etykieta",
      "sourceId": 2
    }
  }
}
```

`sourceId` = ID węzła, którego elementy mają być iterowane. Może wskazywać na getter (iteracja wszystkich) lub filter (iteracja przefiltrowanych). Patrz sekcja "Iterator sourceId — wyjaśnienie wzorca".

router — rozgałęzienie warunkowe

```
{
  "type": "flow_control",
  "payload": {
    "type": "router",
    "data": []
  }
}
```

Router rozdziela przepływ na kilka równoległych gałęzi. Każda gałąź powinna mieć filtr jako następny węzeł.

if — warunkowe wykonanie

```
{
  "type": "flow_control",
  "payload": {
    "type": "if",
    "data": {
      "conditions": []
    }
  }
}
```

---

action

Wykonuje operację na danych.

create_task — tworzenie zadania

```
{
  "type": "action",
  "payload": {
    "type": "create_task",
    "data": {
      "title": "🚨 Alert: {N.campaignName} — wartość: {N.spend}",
      "description": "Szczegółowy opis z wyrażeniami...",
      "project_id": null,
      "priority": null,
      "label_ids": [],
      "assignee_id": null,
      "job_content": null,
      "due_date": null,
      "parent_id": null,
      "collaborator_ids": [],
      "status": null
    }
  }
}
```

---

ai_agent

Wywołanie agenta AI z dynamicznym promptem.

```
{
  "type": "ai_agent",
  "payload": {
    "type": "ai_agent",
    "data": {
      "agent_id": null,
      "input": "Prompt z wyrażeniami: {N.campaignName}, koszt: {N.spend}..."
    }
  }
}
```

Pola wyjściowe: `output` (surowa odpowiedź AI), `parsed_data` (jeśli włączono parse_response)

> `agent_id` *musi być uzupełniony przez użytkownika — w szablonach ustawiony na* `null`.

Składnia wyrażeń

Wyrażenia dynamiczne używane w polach `title`, `description`, `right_operand_value`, `input` itd.

Referencje do węzłów

```
{N.pole}        — wartość pola z węzła o index=N
{2.campaignName}
{4.spend}
{globals.cpa_target}
```

Funkcje daty

Wyrażenie	Opis	Przykład wyniku
`{NOW()}`	Bieżący timestamp	`2026-03-26T08:00:00`
`{NOW().formatDateTime('Y-m-d')}`	Sformatowana data	`2026-03-26`
`{NOW().formatDateTime('d.m.Y')}`	Data polska	`26.03.2026`
`{NOW().subtractNumberOfDays(7)}`	7 dni temu	—
`{NOW().getFirstDayCurrentMonth()}`	Pierwszy dzień miesiąca	—
`{NOW().getDayOfMonth()}`	Numer dnia (1–31)	`26`
`{NOW().getDaysInMonth()}`	Liczba dni w miesiącu	`31`
`{NOW().formatDateTime('F Y')}`	Nazwa miesiąca i rok	`March 2026`
`{NOW().formatDateTime('m')}`	Numer miesiąca	`03`

Funkcje matematyczne

Funkcja	Opis	Przykład
`{divide(a, b)}`	Dzielenie	`{divide({4.cost},{4.conversions})}` → CPA
`{multiply(a, b)}`	Mnożenie	`{multiply({globals.budget},0.1)}`
`{subtract(a, b)}`	Odejmowanie	`{subtract({globals.budget},{4.cost})}`
`{add(a, b)}`	Dodawanie	`{add({4.cost},{5.cost})}`
`{percentageChange(a, b)}`	Zmiana %	`{percentageChange({3.conversions},{5.conversions})}`
`{monthlyDeviation(budget, deviation)}`	Budżet z odchyleniem	`{monthlyDeviation({globals.budget},10)}`

Wzorce struktury szablonów

Wzorzec 1: Prosty alert kampanijny (Google Ads)

```
trigger → get_google_campaign → iterator → get_google_ads_metrics → filter → iterator → create_task
```

Węzły: 7. Przykłady: `cpa_campaign_alert`, `roas_drop_alert`, `zero_impression_campaigns`

Wzorzec 2: Prosty alert kampanijny (Microsoft Ads)

```
trigger → get_microsoft_advertising_campaigns → filter → iterator → create_task
```

Węzły: 5. Przykłady: `ms_cpa_campaign_alert`, `ms_budget_protector`. Prostszy bo kampanie MS Ads zawierają już metryki.

Wzorzec 3: Analiza słów kluczowych/fraz

```
trigger → get__keywords / get__search_terms → filter → iterator → create_task
```

Węzły: 5. Przykłady: `non_converting_keywords`, `ms_non_converting_keywords`, `trending_search_terms`

Wzorzec 4: Porównanie dwóch okresów

```
trigger → getter(okres 1) → getter(okres 2) → filter(porównanie) → iterator → create_task
```

Węzły: 6. Przykłady: `roas_drop_alert`, `ms_roas_drop_alert`, `ms_conversion_comparison`

Wzorzec 5: Router — wiele gałęzi

```
trigger → getter → iterator → getter metryk → router → [filter A → iterator → create_task A, filter B → iterator → create_task B
```

Węzły: 12. Przykłady: `budget_deviation`

Wzorzec 6: AI Agent

```
trigger → getter → ai_agent → create_task
```

Węzły: 4. Przykłady: `ai_weekly_report`, `campaign_intelligence_scoring`

Wzorzec 7: Sekwencyjny podwójny getter + AI

```
trigger → getter_A → getter_B → ai_agent → create_task
```

Węzły: 5. Getter B ma `parent_id` równy ID gettera A — obaj działają sekwencyjnie, nie w pętli. AI może referować pola z obu: `{2.field}` (getter A) i `{3.field}` (getter B). Przykłady: `monthly_cross_platform_summary`, `master_account_audit`, `competitor_gap_analysis`

Wzorzec 8: Analiza treści reklam (ad copy)

```
trigger → get_google_ads_ad_copy → filter(adStatus=ENABLED) → ai_agent → create_task
```

Węzły: 5. Getter zwraca pełne teksty reklam (headline1/2/3, description1/2, finalUrl). Filtr ogranicza do aktywnych reklam. AI analizuje jakość copy lub dopasowanie do kampanii. Przykłady: `ad_copy_quality_scorer`, `ad_copy_message_audit`

Status szablonów — produkcja (2026-03-30)

Po pełnym cyklu testów v1/v2/v3 + naprawach.

Status	Liczba	Opis
✅ Gotowe	30	Bez znanych błędów funkcjonalnych
❌ Zablokowane — błąd logiczny	4	Wymagają naprawy kodu
❌ Zablokowane — weryfikacja	4	Iterator sourceId / pole DTO niezweryfikowane

Zablokowane — błąd logiczny (nie wypuszczać)

Cztery szablony mają zidentyfikowane błędy uniemożliwiające prawidłowe działanie.

`visibility_drop_alert`

Co robi: Monitoruje tygodniowo pozycje fraz w Phrase Monitor. Gdy domena nie jest widoczna w top N pozycjach, tworzy task alertujący.

Przepływ: trigger (co poniedziałek 08:00) → getter `get_phrase_monitor_competitors` (node 2) → filter (node 3, `parent_id: 2`) → iterator (node 4, `sourceId: 3`, `parent_id: 3`) → create_task (node 5)

Bug — nazwy pól w filtrze: Węzeł filter (node 3) używa pól:

- `isMyResult` — warunek: `= true`
- `rankAbsolute` — warunek: `> {globals.max_position}`

Problem: nieznane czy DTO `get_phrase_monitor_competitors` zwraca te pola dokładnie w tej formie (camelCase). Jeśli zwraca snake_case (`is_my_result`, `rank_absolute`) lub inaczej, filtr zawsze zwróci 0 wyników i żaden task nigdy nie zostanie utworzony.

Jak naprawić:

- Sprawdzić w kodzie silnika (`WorkflowGetterType.php` lub odpowiedni DTO) jakie nazwy pól zwraca `get_phrase_monitor_competitors`
- Zaktualizować `left_operand_value` w warunkach filtra do poprawnych nazw
- Przetestować na koncie z aktywnym Phrase Monitor

Ryzyko: Niskie (brak destruktywnego działania — w najgorszym razie tworzy 0 tasków lub za dużo tasków)

`inventory_aware_alert`

Co robi: Miał alertować o produktach z wysoką liczbą konwersji, które są out-of-stock — żeby priorytetyzować uzupełnienie stanów.

Przepływ: trigger (codziennie 08:00) → getter `get_google_merchant_center_top_products` (node 2) → filter (node 3, `parent_id: 2`) → iterator (node 4, `sourceId: 3`, `parent_id: 3`) → create_task (node 5)

Bug — getter zwraca złe dane: Getter `get_google_merchant_center_top_products` zwraca bestsellery z raportu rynkowego GMC, **nie** dane własnego inwentarza. Filtr odwołuje się do pól:

- `conversions` (warunek: `> {globals.min_conversions}`) — pole nie istnieje w tym getterze
- `availability` (warunek: `contains "out_of_stock"`) — pole nie istnieje w tym getterze

Cały szablon opiera się na błędnym założeniu co do danych tego gettera. Task w create_task odwołuje się też do `{4.title}`, `{4.offerId}`, `{4.availability}`, `{4.conversions}`, `{4.price}` — wszystkie nieistniejące pola.

Jak naprawić — opcja A (preferowana): Zmienić getter na `get_google_merchant_center_performance_info`, który zwraca dane wydajności własnych produktów, a następnie:

- Zweryfikować czy ten getter zwraca `availability` i `conversions`
- Dostosować nazwy pól w filtrze i create_task do rzeczywistego DTO

Jak naprawić — opcja B: Jeśli żaden istniejący getter nie zwraca dostępności produktów + metryk — wymagane dodanie nowego gettera w silniku.

Ryzyko: Średnie (getter zwróci dane, ale filtr zawsze zwróci 0 wyników — szablon "działa" ale nic nie robi)

`ms_seasonal_ad_copy_check`

Co robi: Miał sprawdzać miesięczny compliance reklam Microsoft Ads przez AI — weryfikować czy copy zawiera właściwe przekazy dla aktualnego sezonu.

Przepływ: trigger (co miesiąc, pierwszy dzień, 08:00) → getter `get_microsoft_advertising_campaigns` (node 2) → filter (node 3, `spend > 0`) → iterator (node 4, `sourceId: 3`) → ai_agent (node 5) → create_task (node 6)

Bug — getter nie zwraca tekstów reklam: Getter `get_microsoft_advertising_campaigns` zwraca dane kampanii (budżet, metryki, status — `CampaignDTO`), **nie** treści reklam (headline, description). Prompt AI agenta (node 5) operuje na `{2.campaignName}`, `{2.spend}`, `{2.clicks}`, `{2.ctr}` — te pola kampanii istnieją, ale AI jest proszony o ocenę copy sezonowego **na podstawie samej nazwy kampanii**, co jest nieprecyzyjne i podatne na halucynacje.

Opis szablonu obiecuje analizę tekstów reklam, ale w rzeczywistości AI ocenia tylko nazwę kampanii i jej metryki — bez dostępu do faktycznych headline/description.

Jak naprawić: Zmienić getter na dedykowany getter reklam Microsoft Ads lub dodać drugi getter po iteratorze. Kandydaci (do weryfikacji):

- `get_microsoft_advertising_ad_groups` — jeśli zwraca treści reklam
- Jeśli nie istnieje — wymagane dodanie `get_microsoft_advertising_ad_copy` analogicznego do `get_google_ads_ad_copy`

Prompt AI agenta musi zostać dostosowany do pól rzeczywistego gettera zawierającego treści reklam.

Ryzyko: Wysokie (szablon wykona się, ale AI agent ocenia kampanie bez dostępu do treści reklam — może generować błędne/halucynacyjne oceny sezonowości)

`ms_budget_deviation`

Co robi: Miał alertować gdy miesięczny spend Microsoft Ads odbiega od planu budżetu o więcej niż X%.

Przepływ: trigger (codziennie 20:00) → getter `get_microsoft_advertising_campaigns` (node 2, zakres: od początku miesiąca do dziś) → filter (node 3, `spend > monthlyDeviation(budget, deviation)`) → iterator (node 4, `sourceId: 3`) → create_task (node 5)

Bug 1 — brak gałęzi underspend: Szablon wykrywa tylko overspend (`spend > próg górny`). Brak symetrycznej gałęzi dla underspend (`spend < próg dolny`). To pół funkcjonalności — klient nie dostanie alertu gdy kampania drastycznie nie wydaje budżetu.

Bug 2 — potencjalne dzielenie przez zero w opisie taska: Opis taska (node 5) zawiera wyrażenie: `{divide({subtract({globals.budget}, {2.spend})}, {subtract({NOW().getDaysInMonth()}, {NOW().getDayOfMonth()})})}` — na ostatni dzień miesiąca `getDaysInMonth() - getDayOfMonth() = 0`, co powoduje DivisionByZeroError w silniku wyrażeń dla pola "Rekomendowany dzienny budżet".

Jak naprawić:

- Dodać guard w create_task: renderować "Rekomendowany dzienny budżet" tylko gdy `getDaysInMonth() > getDayOfMonth()`
- Dodać drugą gałąź (drugi filter + iterator + create_task) dla underspend
- Opcjonalnie: zweryfikować zachowanie funkcji `monthlyDeviation()` w silniku — czy przyjmuje `globals.budget = 0` bez błędu

Ryzyko: Wysokie (DivisionByZeroError może crashować silnik workflow na ostatni dzień miesiąca)

Zablokowane — wymagają weryfikacji silnika

Cztery szablony mają niejasności wymagające testu na działającym środowisku lub wglądu w kod silnika.

`cpa_campaign_alert`, `roas_drop_alert`, `zero_impression_campaigns`

Wspólny problem — iterator sourceId: W tych trzech szablonach drugi iterator (ten po filtrze) ma `sourceId` wskazujące na getter metryk, a nie na węzeł filtra.

Dokładne wartości per szablon:

`cpa_campaign_alert`:

```
getter kampanii(id:2) → iterator(id:3, sourceId:2) → getter metryk(id:4) → filter(id:5, parent_id:4) → iterator(id:6, sourceId:4
```

Iterator id:6 ma `parent_id: 5` (filter) ale `sourceId: 4` (getter metryk).

`roas_drop_alert`:

```
getter kampanii(id:2) → iterator(id:3, sourceId:2) → getter bieżący(id:4) → getter poprzedni(id:5) → filter(id:6, parent_id:5) →
```

Iterator id:7 ma `parent_id: 6` (filter) ale `sourceId: 4` (getter bieżących metryk).

`zero_impression_campaigns`:

```
getter kampanii(id:2) → iterator(id:3, sourceId:2) → getter metryk(id:4) → filter(id:5, parent_id:4) → iterator(id:6, sourceId:4
```

Iterator id:6 ma `parent_id: 5` (filter) ale `sourceId: 4` (getter metryk).

Pytanie do weryfikacji: Czy silnik workflow, gdy iterator ma `sourceId: 4` (getter metryk), ale `parent_id: 5` (filter) — iteruje po danych z gettera czy z filtra?

- Jeśli iteruje po rodzicu (`parent_id`, czyli filtr) → działa poprawnie

- Jeśli iteruje po `sourceId` (getter) → bug (przetwarza wszystkie kampanie z metrykami, ignoruje filtr)

Jak zweryfikować: Uruchomić jeden z tych szablonów na koncie testowym z kampaniami, które powinny i nie powinny przejść filtra. Sprawdzić ile tasków zostało utworzonych — jeśli więcej niż kampanii spełniających warunek filtra, `sourceId` wygrywa.

Jak naprawić (jeśli bug): Zmienić `sourceId` drugiego iteratora na id węzła filtra (wzorzec B — jak w szablonach MS Ads).

`campaign_intelligence_scoring`

Co robi: Tygodniowy AI scoring kampanii Google Ads — AI ocenia każdą kampanię w skali 1-10 na podstawie metryk z ostatnich 30 dni i tworzy task z raportem.

Przepływ: trigger (co poniedziałek 08:00) → getter `get_google_ads_performance` (node 2, resource: campaign) → iterator (node 3, `sourceId: 2`) → ai_agent (node 4) → create_task (node 5)

Problem — nazwa pola ROAS: Prompt AI agenta (node 4) odwołuje się do `{2.returnOnAdSpend}`. Getter `get_google_ads_performance` może zwracać to pole jako `roas` (krótka forma) lub pod inną nazwą.

Jak zweryfikować: Sprawdzić DTO gettera `get_google_ads_performance` w kodzie silnika — jaką nazwą klucza zwracane jest ROAS.

Jak naprawić (jeśli pole nazywa się inaczej): Zaktualizować referencję w prompcie AI: `{2.returnOnAdSpend}` → `{2.roas}` lub właściwa nazwa. Pozostałe pola w prompcie (`{2.campaignName}`, `{2.cost}`, `{2.clicks}`, `{2.impressions}`, `{2.conversions}`, `{2.conversions_value}`, `{2.ctr}`, `{2.costPerConversion}`) wymagają analogicznej weryfikacji.

Ryzyko: Niskie-średnie (AI dostanie pustą wartość dla ROAS i wygeneruje scoring bez tej metryki — może dawać błędne wyniki ale nie crashuje)

Iterator sourceId — wyjaśnienie wzorca

Dwa poprawne wzorce użycia:

Wzorzec A — iterator przed filtrem (iteruje wszystkie elementy gettera):

```
getter(N) → iterator(sourceId: N) → getter_metryk → filter → create_task
```

Stosowany gdy potrzebujesz pobrać dodatkowe dane per element. Przykład: `budget_protector`.

Wzorzec B — iterator po filtrze (iteruje tylko przefiltrowane elementy):

```
getter(N) → filter(M) → iterator(sourceId: M) → create_task
```

Stosowany gdy filtr jest ostatnim krokiem przed akcją. `sourceId` wskazuje na filtr (M), nie getter (N). Przykłady: `ms_cpa_campaign_alert`, `ms_roas_drop_alert`, `visibility_drop_alert`.

Problem: Część szablonów Google Ads używa wzorca B, ale `sourceId` wskazuje błędnie na getter (N). Nieznane zachowanie silnika — może iterować po wszystkich elementach zamiast przefiltrowanych.

Dodawanie nowego szablonu

Krok 1 — Sprawdź wykonalność

Przed tworzeniem szablonu zweryfikuj:

- Czy wymagane gettery istnieją w `WorkflowGetterType.php`?
- Czy potrzebne pola są dostępne w odpowiednim DTO?
- Czy akcje (`create_task`, `add_negative_keywords`) są włączone w fabryce?

Krok 2 — Utwórz plik JSON

```
# Lokalizacja
sembot-laravel/workflow-templates/nazwa_szablonu.json
sembot-angular/src/assets/workflow-templates/nazwa_szablonu.json
```

Plik musi być identyczny w obu lokalizacjach.

Krok 3 — Zarejestruj w const.ts (Angular)

```
// sembot-angular/src/app/dashboard/workflows/workflow-templates/workflow-templates.const.ts
{
  slug: 'nazwa_szablonu',          // musi odpowiadać nazwie pliku JSON (bez .json)
  name: 'Wyświetlana Nazwa',
  description: 'Opis szablonu.',
  category: 'alerts | optimization | reporting | intelligence | compliance',
  icon: 'material_icon_name',
  integrations: ['Google Ads'],    // z dostępnych: 'Google Ads', 'Microsoft Advertising', 'Google Merchant Center', 'Phrase Mon
},
```

Krok 4 — Weryfikacja

```
# Sprawdź czy plik jest dostępny statycznie
curl http://localhost:4200/assets/workflow-templates/nazwa_szablonu.json
```

Sprawdź czy slug pasuje do pliku

```
grep "slug: 'nazwa_szablonu'" workflow-templates.const.ts
```

Dostępne gettery — pełna lista

Getter type	Integracja	DTO	Opis
`get_google_campaign`	Google Ads	—	Lista kampanii (statyczne dane)
`get_google_ads_metrics`	Google Ads	—	Metryki kampanii/słów kluczowych za okres
`get_google_ads_performance`	Google Ads	—	Wyniki z możliwością filtrowania zasobów
`get_google_ads_customer`	Google Ads	—	Dane konta Google Ads (poziom klienta)
`get_google_ads_ad_copy`	Google Ads	`GoogleAdsAdCopyDTO`	Treści reklam (headline1/2/3, description1/2, finalUrl)
`get_google_merchant_center_top_products`	GMC	—	Top produkty z raportów GMC
`get_google_merchant_center_performance_info`	GMC	—	Dane wydajności GMC
`get_google_analytics_4_source_medium_info`	GA4	—	Źródła ruchu GA4
`get_visibility_for_site`	Phrase Monitor	—	Widoczność SEO serwisu
`get_phrase_monitor_competitors`	Phrase Monitor	—	Pozycje fraz (własne + konkurenci)
`get_microsoft_advertising_campaigns`	Microsoft Ads	`CampaignDTO`	Kampanie z metrykami
`get_microsoft_advertising_ad_groups`	Microsoft Ads	—	Grupy reklam z metrykami
`get_microsoft_advertising_search_terms`	Microsoft Ads	`SearchTermDTO`	Frazy wyszukiwania
`get_microsoft_advertising_keywords`	Microsoft Ads	`KeywordDTO`	Słowa kluczowe z metrykami
`get_microsoft_advertising_products`	Microsoft Ads	—	Produkty Microsoft Ads

Istniejące szablony — indeks

Google Ads (alerty)

Slug	Wzorzec	Status	Opis
`cpa_campaign_alert`	1	❌	Alert gdy CPA > cel
`roas_drop_alert`	4	❌	Alert gdy ROAS < cel (porównanie tygodniowe)
`zero_impression_campaigns`	1	❌	Kampanie bez wyświetleń
`budget_protector`	1	✅	Alert przekroczenia budżetu dziennego
`budget_deviation`	5	✅	Odchylenie od planu budżetu miesięcznego

Google Ads (optymalizacja)

Slug	Wzorzec	Status	Opis
`non_converting_keywords`	3	✅	Słowa kluczowe bez konwersji
`trending_search_terms`	3	✅	Rosnące frazy wyszukiwania

Inne integracje

Slug	Wzorzec	Integracja	Status	Opis
`microsoft_ads_waste_finder`	3	Microsoft Ads	✅	Frazy bez konwersji
`ms_ads_winners`	3	Microsoft Ads	✅	Frazy wygrywające
`visibility_drop_alert`	3	Phrase Monitor	❌	Spadek własnej widoczności
`price_competitiveness_alert`	3	GMC	✅	Niekonkurencyjne ceny
`inventory_aware_alert`	3	GMC	❌	Produkty out-of-stock z budżetem

Raporty i Intelligence

Slug	Wzorzec	Status	Opis
`ai_weekly_report`	6	✅	AI tygodniowy raport Google Ads
`conversion_comparison`	4	✅	Porównanie konwersji 30/60 dni
`campaign_intelligence_scoring`	6	❌	AI scoring kampanii Google Ads
`pmax_analysis`	6	✅	AI analiza kampanii PMax
`seasonal_ad_copy_check`	6	✅	AI compliance check reklam

AI Intelligence — nowe szablony (2026-03)

Slug	Wzorzec	Trigger	Status	Opis
`daily_ppc_brief`	6	Codziennie 08:00	✅	AI podsumowanie wyników kampanii z minionego tygodnia
`weekly_opportunity_scout`	6	Poniedziałek	✅	AI skanuje frazy wyszukiwania pod kątem niebidowanych tematów
`search_terms_clustering`	6	1. miesiąca	✅	AI grupuje frazy wyszukiwania według intencji i proponuje nowe grupy reklam
`monthly_cross_platform_summary`	7	1. miesiąca	✅	Google Ads + Microsoft Ads razem — AI pisze miesięczne podsumowanie cross-platform
`keyword_quality_audit`	6	Poniedziałek	✅	AI ocenia efektywność słów kluczowych, flaguje te z dużym spędem bez konwersji
`master_account_audit`	7	Poniedziałek 07:00	✅	Kampanie + frazy wyszukiwania razem — AI identyfikuje 2–3 najważniejsze rzeczy do naprawy
`competitor_gap_analysis`	7	Poniedziałek	✅	Phrase Monitor + Google Ads keywords — AI wskazuje frazy konkurentów na których nie bidiujesz
`low_ctr_detective`	6	Poniedziałek	✅	AI analizuje kampanie z dużą liczbą wyświetleń ale niskim CTR (<2%)
`ad_copy_quality_scorer`	8	1. miesiąca 08:00	✅	AI ocenia jakość nagłówków i opisów reklam (skala 1–5), wskazuje co przepisać
`ad_copy_message_audit`	8	1. miesiąca 09:00	✅	AI sprawdza czy copy reklam pasuje do intencji kampanii — wyłapuje generyczne nagłówki i brak USP

Microsoft Ads — pełna biblioteka

Slug	Wzorzec	Status	Opis
`ms_cpa_campaign_alert`	2	✅	Alert gdy CPA > cel
`ms_roas_drop_alert`	2+4	✅	Alert gdy ROAS < cel
`ms_zero_impression_campaigns`	2	✅	Kampanie bez wyświetleń
`ms_budget_protector`	2	✅	Alert przekroczenia budżetu
`ms_budget_deviation`	2	❌	Odchylenie budżetu miesięcznego
`ms_non_converting_keywords`	3	✅	Słowa kluczowe bez konwersji
`ms_trending_search_terms`	3	✅	Rosnące frazy
`ms_ai_weekly_report`	6	✅	AI tygodniowy raport
`ms_conversion_comparison`	4	✅	Porównanie konwersji
`ms_campaign_intelligence_scoring`	6	✅	AI scoring kampanii
`ms_seasonal_ad_copy_check`	6	❌	AI compliance check

Znane ograniczenia

Ograniczenie	Wpływ	Status
`add_negative_keywords` wyłączone w backendzie	Szablony nie mogą auto-pauzować słów kluczowych	Tworzy task zamiast
`get_google_ads_ad_copy` nie zwraca asset-level danych (RSA pinning)	Getter zwraca przypisane teksty, nie wszystkie assety z konfiguracją pinowania	Akceptowalne dla analizy copy
`get_phrase_monitor_competitors` bez filtrów daty	Brak porównania tygodniowego widoczności	Przeformułowane na "własna widoczność"
`get_microsoft_advertising_campaigns` wymaga dat	Nie zwraca listy bez `start_date` / `end_date`	Daty dynamiczne przez `{NOW()}`
AI agent wymaga `agent_id`	Szablony AI wymagają konfiguracji agenta przez użytkownika	Dokumentowane w opisie szablonu
`get_phrase_monitor_competitors` pola w filtrach	Filtry wymagają camelCase (`rankAbsolute`, `isMyResult`), nie snake_case	Weryfikacja wymagana — brak testu E2E
`get_google_merchant_center_top_products` — brak danych inwentarza	Getter zwraca bestsellery rynkowe, nie stan własnego magazynu	`inventory_aware_alert` wymaga przebudowy
`ms_seasonal_ad_copy_check` — zły getter	`get_microsoft_advertising_campaigns` nie zwraca tekstów reklam	Wymaga gettera `get_microsoft_advertising_ad_groups` lub equivalent
`budget_deviation` / `ms_budget_deviation` — dzielenie przez 0	`divide(remaining, daysLeft)` = 0 na ostatni dzień miesiąca	Edge case, niski priorytet